Context-Coupled Weight Placement: A Correctness Failure in Local LLM Offloading on Memory-Constrained Consumer GPUs

Neelesh Padmanabh
Ramer Project

Trisha Janath
Ramer Project

***Abstract*—Weight placement is a critical decision in on-device large language model (LLM) inference, determining how model layers are split between GPU and system memory when a model exceeds GPU capacity. In this paper, we conduct an empirical study of how this decision affects inference performance on memory-constrained consumer hardware. We examine the static, estimate-based placement used by widely deployed inference engines and compare it against placements selected by direct measurement on one 8 GB consumer GPU. We find that default placement is consistently conservative: across three models and quantization formats, measured placement improves decode throughput by 6.7% to 402%, a gap that varies too widely to be corrected by any fixed safety margin. Our analysis further reveals two challenges that static estimation cannot address. The first is *context coupling*: a placement chosen at one context length becomes invalid at a longer one as the key-value (KV) cache grows, which reduced throughput from 8.71 to 1.47 tokens/s across consecutive calls in one session. The second is *unpredictable oversubscription*: once physical GPU memory is exceeded, the driver silently spills into system RAM, causing non-deterministic slowdowns. Building on these findings, we present Ramer, a runtime layer that selects placements by measurement, keys them to context length, re-validates them at run time, and coordinates which model remains loaded when multiple agents share one GPU. Through end-to-end experiments we show that Ramer reduces the median completion time of four concurrent agents from 210.6 s to 117.6 s and completes an externally verified coding task 38% faster, while providing no benefit when tool latency dominates. Our findings indicate that weight placement on consumer hardware should be measured rather than estimated.**

***Index Terms*—on-device inference, edge AI, large language models, GPU memory management, model offloading, autotuning, LLM agents.**

I. INTRODUCTION

Deploying large language models (LLMs) on-device, on consumer and edge hardware owned by the user, has become common for privacy-sensitive, offline, latency-sensitive, and cost-sensitive workloads. This shift has been enabled by aggressive post-training quantization [1]–[4], efficient CPU and GPU kernels, and packaging tools that reduce deployment to a single command. Unlike datacenter accelerators, such devices have limited GPU memory: in the Steam hardware survey of August 2026, a large if gaming-skewed census, over 60% of GPUs have 12 GB or less and 8 GB configurations alone account for roughly a quarter [5]. This is often less than even a quantized model requires. 16 GB overtook 8 GB as the single most common configuration only in mid-2026, and quantized model footprints have grown at least as quickly as installed memory, so the constrained regime is not disappearing. Configuration on these devices, however, has received far less attention than compression or kernels. Datacenter serving systems are tuned continuously against their own workloads, whereas a local installation is typically configured once, by a heuristic, using a hardware profile that is inferred rather than measured.

In this paper, we study one such configuration decision: how many transformer layers to place in GPU memory when a model does not fit entirely. This decision is unavoidable on memory-constrained hardware and is made automatically by every widely used inference engine, yet its quality is rarely evaluated on the machine where it is applied. The maintainers of llama.cpp, which underpins most consumer deployments, acknowledge this directly. They note that downstream tools automate memory allocation with rough heuristics that tend to be inaccurate. Because overestimating available memory risks out-of-memory failures, these heuristics are conservative and leave potential performance unused [6]. Recent work within llama.cpp addresses this by iteratively fitting memory allocation inside the engine at load time [6]. However, it optimizes for memory fit rather than measured throughput, and does not re-validate a placement once it is in use.

Specifically, we quantify the cost of this conservatism on an 8 GB consumer GPU, examine why it cannot be corrected by a better safety margin, and describe a runtime layer that replaces estimation with measurement. Our analysis shows that the problem extends beyond efficiency. Because model weights and the key-value (KV) cache share GPU memory, a placement is valid only at the context length for which it was chosen, a property we term *context coupling*. In addition, exceeding physical GPU memory causes the driver to spill into system RAM with effects that vary from run to run, which we term *unpredictable oversubscription*. Neither problem can be resolved by a static estimate.

Our contributions are as follows:

- **Characterization of the placement gap.** We show that measured placement improves decode throughput over a production engine's default by 6.7% to 402%, depending on model and quantization format (Section III-A). Sweeping the parameter by hand is established practice [7]; our claim is the magnitude and variance of the gap, not the act of measuring.
- **Context coupling.** We show that placement validity depends on context length, making it a correctness issue

rather than a tuning one: reusing a placement at a longer context reduced throughput 5.9-fold (Section III-B).

- **Unpredictable oversubscription.** We measure GPU memory oversubscription under the Windows driver. We report the negative result that exceeding physical memory is necessary but not sufficient to cause a slowdown, which rules out any static defense (Section III-C).
- **Design of Ramer.** We present a probe-and-cache placement engine with a headroom-aware tie-break, run-time re-validation, and a single-machine multi-agent scheduler (Section IV). Residency itself is already managed on the desktop [8], [9]; our contribution is the policy and its ablation.
- **End-to-end evaluation.** We evaluate Ramer through unmodified third-party agents, verifying outcomes with test suites and version-control diffs rather than model self-reports. We report three negative results for our own method (Section VI).

II. BACKGROUND AND RELATED WORK

LLM text generation has two phases. In the prefill phase, the model processes the full prompt in a single pass; this phase is limited mainly by computation. In the decode phase, the model generates one token at a time and must read its weights from memory for every token; this phase is limited mainly by memory bandwidth. Because decode accounts for most of the delay a user experiences, where the weights are stored largely determines generation speed.

This matters most on edge and consumer devices, where GPU memory is much faster than system memory but also much smaller. The GPU used in this paper provides a nominal 448 GB/s of memory bandwidth and connects to the host over a PCIe 5.0 ×8 link [10], while its dual-channel DDR5 system memory is rated at roughly 83 GB/s; these are specification figures, not measurements of our own. When a model does not fit on the GPU, inference engines run the remaining layers on the CPU from system memory, over a path with roughly one-fifth the bandwidth. The number of layers placed on the GPU is therefore the main factor in decode speed. Edge devices are additionally limited by energy and heat, which can make sustained inference impractical [11]; we do not instrument power or temperature, and note this in Section VII-B. The rest of this section reviews related work, grouped by the resource each approach targets.

A. On-Device LLM Inference

Several systems run LLMs directly on edge devices. MLC-LLM compiles models for a wide range of hardware, including mobile GPUs and web browsers [12]. llm.npu uses the neural processing units (NPUs) in smartphones to speed up prompt processing [13]. PowerInfer-2 [14] and EdgeMoE [15] run large models on phones by loading parts of the model from storage only when needed, and Apple's approach runs models up to twice the size of available memory by reading weights from flash storage [16]. On desktop GPUs, PowerInfer keeps the most frequently used parts of the model on the GPU and runs the rest on the CPU, reporting up to 11.69× the speed of llama.cpp on a ReLU-based 40B model [17]. MCAP is closest to us in framing: it profiles on the target device at load time and uses the result to assign each layer a residency tier across GPU, RAM and SSD [18]. What it profiles is layer *importance*, a sensitivity to precision reduction, and its signal is deliberately hardware-independent, cached per architecture and reported byte-identical across two silicon generations; ours is a throughput measurement specific to one machine's memory state. Context length does not enter MCAP, and we regard the two signals as complementary. The MELT study reports that on-device inference is limited chiefly by memory, energy and heat [11]. All of these systems introduce new runtimes, kernels, or hardware support. Ramer takes a different approach: it adds no new execution mechanism, and instead tunes the placement setting that existing engines already provide.

B. Model Compression

Quantization reduces the size of model weights by storing them at lower precision, and is the most common way to fit LLMs on local hardware [1]. Widely used methods include GPTQ [2], AWQ [3], and SmoothQuant [4]. Quantization determines how much memory a model needs, but not where its weights should be placed. It does, however, change the per-layer memory size that any placement decision must take into account.

C. Decoding Acceleration

Speculative decoding speeds up generation by using a small draft model to propose several tokens that the main model then verifies, typically achieving a 1.5–2.5× speedup [19]. Dovetail adapts this technique to consumer machines with both a CPU and a GPU [20]. However, the draft model needs its own GPU memory, which competes with the main model when GPU memory is already the limiting factor. These methods also do not address how many layers to place on the GPU.

D. KV-Cache Management

During generation, the model stores intermediate results for previous tokens in a key-value (KV) cache, which grows with the length of the context. PagedAttention manages this cache in fixed-size blocks, reducing wasted memory and improving server throughput by 2–4× [21]. Other methods discard less important cache entries or compress the cache [22], [23], and LMCache stores it across CPU memory, disk, and remote servers so it can be reused between requests [24]. These techniques mainly benefit servers handling many requests, which a single user does not generate. One property, however, is central to this paper: the KV cache and the model weights share the same GPU memory.

E. Weight Placement and Offloading

Several systems move model weights between GPU memory, system memory, and disk. ZeRO-Inference streams layers from system memory or disk [25], and FlexGen chooses a placement by solving an optimization problem, running a

175-billion-parameter model on a single 16 GB GPU [26]. Both are designed for large batches of requests, and their placements may perform poorly for a single interactive user. For mixture-of-experts (MoE) models, which activate only part of the model for each token, Fiddler [27], MoE-Gen [28], and MoBiLE [29] split the work between the CPU and GPU.

Consumer inference engines, by contrast, reduce placement to a single setting: the number of layers placed on the GPU, chosen automatically from a fixed memory estimate [6]. This estimate is the focus of our work.

F. Scheduling and Multi-Model Coordination

Orca [30] and Sarathi-Serve [31], [32] improve server throughput by scheduling many requests together, which requires more concurrent requests than a single user produces. Prism allows multiple models to share GPU memory on servers, reclaiming memory from inactive models and giving it to active ones, and has been deployed on more than 10,000 GPUs [33], [34]. ServerlessLLM reduces the time needed to load models on demand in the cloud [35]. These systems target large clusters. On the desktop the problem is not unmanaged either: Ollama's scheduler caps how many models stay resident, queues requests that do not fit, and unloads on a keep-alive timer [9], and llama-swap is a transparent OpenAI-compatible proxy that swaps the active model per request and unloads on a time-to-live [8]. Neither orders callers by priority, admits requests by model affinity, or configures a model automatically. Ramer's scheduler addresses the same competition for GPU memory on a single desktop machine; its contribution lies in the policy, evaluated by ablation in Section VI-D, rather than in a new mechanism.

G. Configuration Tuning

Some recent work improves performance by choosing better settings rather than adding new techniques. Bench360 compares configurations across models, engines, and quantization methods for local deployment [36]. A complementary line removes the search instead of automating it: Qiu et al. predict llama.cpp throughput from GGUF metadata using roofline-shaped predictors, explicitly to reduce the number of candidate runs a user must perform [37]. Their held-out decode error is 13.1% and 14.4% on two unified-memory Apple systems but 36.1% on a discrete NVIDIA GPU, with a prefill baseline reaching 108.2% on that machine, and they conclude that fitted efficiencies are not universal [37]. We read this as independent support for measuring on the class of hardware this paper targets. The work closest to ours is the automatic fitting feature added to llama.cpp in December 2025 [6]. It tests memory allocations when the model loads and reduces memory use until the model fits, reaching 85–90% GPU memory usage. That feature aims to fit the model into memory; Ramer instead measures generation speed, keys the result to context length, re-checks its choice while the model is in use, and coordinates GPU access between multiple agents. Section VII discusses the overlap in detail.

H. Agentic Workloads

Recent studies show that in many AI agent applications, a large share of the total time is spent outside the model, for example waiting on tools and web requests [38], [39]. This limits how much any improvement to model inference can speed up an agent. Section VI-E reports a case where this limit applies to our own system.

I. Positioning

Table I compares Ramer with the most closely related systems. To our knowledge, no existing system combines speed-based placement, context-keyed validity, run-time adjustment, and coordination between multiple models on consumer hardware without modifying the inference engine. Each element individually has prior art; the combination, and the context-keying in particular, is what we claim.

TABLE I. Comparison with closely related systems. *RT adj.*: revises its choice while serving. *Models*: coordinates several models on one device. *Stock engine*: works against an unmodified inference engine.

System	Target	Placement from	RT adj.	Models	Stock engine
FlexGen [26]	1 GPU, batch	Optimization model	No	No	No
PowerInfer [17]	Consumer GPU	Offline profiling	No	No	No
MCAP [18]	Edge / consumer	Load-time importance	No	No	No
Prism [33]	GPU cluster	Memory sharing	Yes	Yes	No
llama.cpp fit [6]	Consumer GPU	Memory estimate	No	No	Built in
llama-swap [8]	Consumer GPU	User config	No	Yes	Yes
Ramer	**Consumer / edge**	**Measured speed, per context**	**Yes**	**Yes**	**Yes**

III. MOTIVATION: CHARACTERIZING THE PLACEMENT GAP

This section measures how well the default placement performs on typical consumer hardware, and identifies the problems that any placement method must handle. Each subsection reports one observation and concludes with the requirement it places on a solution. Section III-E summarizes these requirements, and Section IV describes how Ramer meets them.

Experimental setup. All measurements were taken on a single machine: an NVIDIA RTX 5060 GPU with 8151 MiB of memory (Blackwell architecture), an AMD Ryzen 5 7600 CPU, and 32 GB of system memory, running Windows 11 with Ollama and the llama.cpp inference engine. We use an 8 GB GPU because it is one of the two most common GPU memory sizes among consumer PCs [5], and because it cannot fully hold the 14–32 billion-parameter models commonly used for coding and agent tasks. For each placement, we reloaded the model from scratch before measuring, so that no state

carried over between measurements.

A. The Default Placement Leaves Performance Unused

We measured decode speed for Qwen2.5-14B-Instruct (Q4_K_M quantization; 48 layers; 9.0 GB on disk) at a 4096-token context, varying the number of GPU layers from 0 to 48 and comparing each result with the engine's automatic choice (Fig. 1).

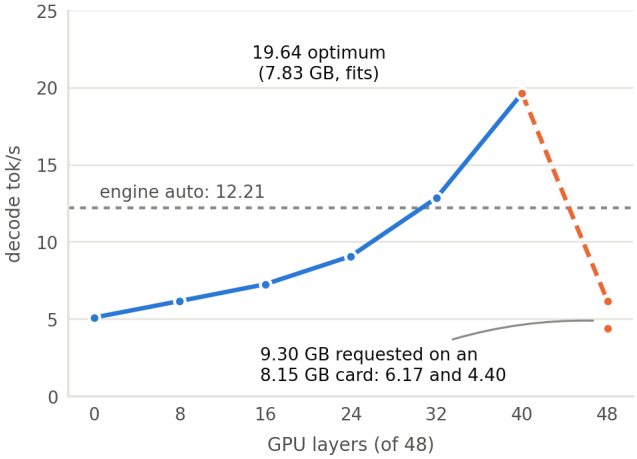


Fig. 1. Decode speed versus the number of GPU layers for Qwen2.5-14B (Q4_K_M) on an 8 GB GPU. The engine's automatic placement uses 6.20 GB and reaches 12.21 tokens/s. The best measured placement, 40 layers, uses 7.83 GB and reaches 19.64 tokens/s while still fitting in GPU memory. Placing all 48 layers requires 9.30 GB and causes a sharp slowdown.

We make three observations. First, the engine's offloading itself works well: from 0 to 40 layers, both decode and prefill speed increase steadily as more layers are placed on the GPU. Second, the automatic placement is overly cautious. It leaves 1.63 GB of GPU memory unused, and the best measured placement is 61% faster. Third, placing all 48 layers on the GPU causes a sharp slowdown, to 6.17 and 4.40 tokens/s on two identical runs; the second run is slower than the 5.11 tokens/s achieved with no GPU at all.

This gap is not a fixed amount that a better safety margin could correct. Table II shows that it varies widely across models and quantization formats, and Fig. 2 plots the same comparison.

TABLE II. Decode speed (tokens/s) with the engine's automatic placement and with measured placement.

Model	Quantization	Automatic	Measured	Gain
Qwen2.5-14B	Q4_K_M	12.2	19.6	+61%
Qwen2.5-32B	Q2_K	3.1	3.3	+6.7%
Qwen3.8-27B	IQ2_XXS	5.9	29.6	+402%

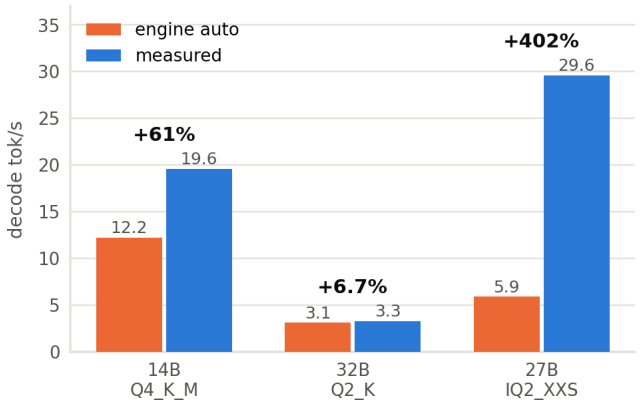


Fig. 2. Automatic versus measured placement across three models and quantization formats on the same machine.

For the 27B model, the automatic placement left about 2.3 GB of GPU memory unused, even though measurement showed that the entire model fits on the GPU. The 32B model shows the opposite case: at 12.3 GB, it cannot fit on the GPU under any placement, and measurement correctly finds little room for improvement. Across the three models, the gain varies by a factor of 60, which suggests that the engine's memory estimate is least accurate for quantization formats it was not designed around. Independent work fitting a throughput predictor from model metadata reaches a compatible conclusion, transferring well between unified-memory systems but degrading on a discrete consumer GPU [37].

Requirement R1: Because the gap cannot be predicted from the model alone, placement should be chosen by measuring performance on the target machine.

B. Context Coupling: The Best Placement Depends on Context Length

The model weights and the KV cache share GPU memory, and the KV cache grows with the context length. The largest placement that fits therefore depends on the context length as well as the model. This makes placement a correctness issue rather than only a tuning issue. In one agent task that involved reading source files, a placement chosen at a 4096-token context was reused at an 8192-token context. The first model call ran at 8.71 tokens/s; the second ran at 1.47 tokens/s, a 5.9× slowdown. We confirmed the cause directly. The same placement used 7.27 GB at 4096 tokens and 7.89 GB at 8192 tokens, because the KV cache doubled in size, and the larger amount exceeded the memory available on the GPU. Choosing the placement again at the correct context length reduced the task time from 85.9 s to 24.1 s.

That incident is a single observation. To test the dependence directly, we swept context length with the model and hardware fixed, choosing a fresh placement at each point (Fig. 3). The selected layer count moves, from 37 at 2048 tokens to 33 at 6144 and 8192, so context is an input to the placement and not merely a constraint to be checked against one computed earlier. Re-selecting per context keeps a 16.2% to 35.5% advantage over the automatic placement at every context tested, with no sign of thinning as context grows.

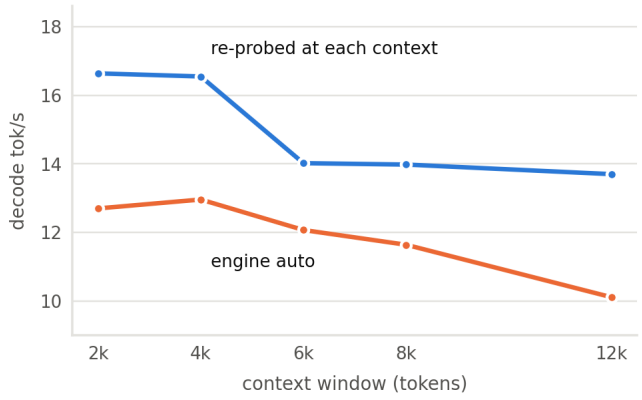


Fig. 3. Context-length sweep for Qwen2.5-14B (Q4_K_M). Re-selecting per context holds 13.7 to 16.6 tokens/s while the automatic placement declines from 12.70 to 10.11 tokens/s. Inferred KV growth over this range is 1.54 GB and smooth, so the divergence comes from the placement policy and not from the cache.

Requirement R2: A placement should be stored and reused only for the context length at which it was measured.

C. Unpredictable Oversubscription

On Windows, since driver version 536.40, a GPU memory allocation that exceeds the available memory does not fail. Instead, the driver silently places the excess in system memory [40]. Accessing data that has been moved this way can be very slow: NVIDIA reports that scattered access to oversubscribed memory can drop to a few hundred kilobytes per second [41]. Our measurements during the 48-layer run in Section III-A are consistent with this. GPU memory usage stayed fixed at 7758 MiB, GPU utilization stayed at 95–100%, and speed varied between identical runs.

However, exceeding GPU memory does not always cause a slowdown. In the context-length experiment of Section III-B, a 37-layer placement at a 12,288-token context reported 8.521 GB of GPU memory use, more than the GPU physically has, yet still ran at 13.93 tokens/s. The same model and layer count had slowed to 1.47 tokens/s at a shorter context that used only 7.89 GB. Exceeding GPU memory is therefore necessary for a slowdown, but not sufficient to cause one, and we found no measurable quantity that predicts which outcome will occur. As a result, a slowdown cannot be ruled out in advance and must instead be detected while the model is running.

Requirement R3: A placement method should keep a margin of free GPU memory where possible, and should check performance at run time so that it can recover when a slowdown occurs.

D. Multiple Agents Competing for One GPU

Users increasingly run several AI agents at the same time on one machine. When four agents shared two models on our GPU, the engine had no information about which request was most important. It switched between models an average of 6.6 times per trial, and the median completion time was 210.6 s. With coordination, the median dropped to 117.6 s (Section VI-D). This is the same problem of models competing for GPU memory that Prism addresses in clusters [33], occurring on a single machine. Desktop engines and proxies do manage residency [8], [9], but without reference to caller priority or model affinity.

Requirement R4: When several agents share one GPU, a placement method should coordinate which model stays loaded, to avoid repeatedly unloading and reloading models.

E. Summary

Table III summarizes the four observations and the requirements they place on a solution. Existing engines meet none of these requirements: they choose a placement once, from an estimate, without considering context length, run-time behavior, or competing agents.

TABLE III. Observations and resulting design requirements.

Observation	Requirement	Addressed in
Default placement is 6.7%–402% slower than measured placement (III-A)	R1: Measure, do not estimate	Sec. IV-A
Best placement changes with context length (III-B)	R2: Store placements per context length	Sec. IV-A
Exceeding GPU memory causes unpredictable slowdowns (III-C)	R3: Keep a memory margin and check at run time	Sec. IV-A, IV-B
Multiple agents cause repeated model switching (III-D)	R4: Coordinate which model stays loaded	Sec. IV-C

IV. DESIGN OF RAMER

Ramer sits between an agent tool and the inference engine. It contributes no offloading mechanism, no kernel and no quantization format. It chooses a configuration for machinery the engine already exposes, keys that choice to context, checks it while running, and applies a residency policy when several agents want the card at once.

A. Context-keyed selection

Algorithm 1 gives the rule. The cache key is the pair of model and context, and that pairing is the whole mechanism behind Section III-B: an entry recorded under one memory pressure cannot match a lookup made under another, so it gets re-selected instead of quietly reused. Entries from a different context simply fail to match and are dropped.

Algorithm 1: Context-keyed placement selection

Input: model m, context c, candidate layer counts G, tolerance δ
Output: chosen layer count g^*
if cache contains (m, c) and entry is fresh **then return** it
for each g in G **do**
  reload m with g layers on GPU at context c
  $T[g], V[g] \leftarrow$ timed bounded decode; reported VRAM
$T^* \leftarrow$ the largest $T[g]$ observed
$g^* \leftarrow$ the g of least $V[g]$ among those with $T[g] \geq T^*(1 - \delta)$
cache $(m, c) \rightarrow (g^*, T[g^*])$; **return** g^*
at run time: time a short generation; **if** it falls well below $T[g^*]$ **then** fall back to the engine's automatic placement and mark the entry stale

Line 6 breaks ties instead of taking an argmax, and it is there because of something that happened to us. On the first context-aware selection the raw argmax chose 41 layers at a reported 8.68 GB, already over the ceiling and already paging, in preference to 33 layers at 7.12 GB, on the strength of 0.04 tokens/s. Noise had bought a zero-headroom placement, and by Section III-C zero headroom is the state from which collapse becomes possible at all. Preferring the smaller footprint among candidates we cannot tell apart costs nothing measurable and removes that whole class of selection.

B. Checking at run time

Line 8 falls straight out of Section III-C. With no pre-run quantity able to predict collapse, the system times a short generation against its cached figure, and on a sustained shortfall it abandons its own selection in favour of the engine's conservative default for that run. Nothing else in the system can respond to the non-deterministic case. We biased it deliberately toward surrendering the optimization rather than defending it.

C. Residency policy for concurrent agents

Given an engine that already manages residency [9] or a proxy that already swaps models [8], what is left open is policy. Ramer funnels concurrent agents through a single chokepoint running two rules. Sticky affinity favours a request for whichever model is already resident and holds a short grace window before paying for a switch. Priority orders callers against each other. Section VI-D pulls the two apart by ablation instead of assuming they cooperate.

V. IMPLEMENTATION

Ramer is about 6000 lines of Python. Hardware detection reports only real measurements. The selection engine keeps its cache on disk, keyed by model and context. An agent-tool layer confines file access to a workspace and gates the dangerous operations behind permission prompts. A proxy speaks both the engine's native protocol and the OpenAI-compatible chat API, so an existing agent changes one base URL and nothing else; the two surfaces differ in whether tool-call arguments arrive as parsed objects or as JSON strings, which the proxy normalizes. Every session closes with a version-control check that compares the files an agent claims to have written against the working tree. That exists because during our own validation a session which wrote nothing at all reported modifying three files.

VI. EVALUATION

A. Protocol

We never simulate a measurement. A quantity we could not measure is missing here instead of modelled. Agentic outcomes are checked from outside, by test suite and version-control diff, never by asking the agent how it did. Single runs are labelled as single runs. Three of the results below go against our own method.

B. Placement and context

Sections III-A and III-B carry the placement results, and two properties of them matter for the mechanism. The selected layer count changes with context, so the cache key is doing work instead of padding the cache with a redundant dimension. And the advantage over the default does not decay as context grows, so this is not an artefact of small contexts where headroom is easy.

C. Overhead

Selection costs one bounded generation per candidate layer count, paid once per model and context and then cached on disk. A full sweep takes a few minutes; every subsequent run of that model at that context pays nothing. The run-time check in Algorithm 1 line 8 is a single short generation at the head of a session. Neither cost is on the per-token path.

D. Residency policy ablation

Four agents with different models, tasks and workspaces ran concurrently under four conditions, five trials each (Fig. 4).

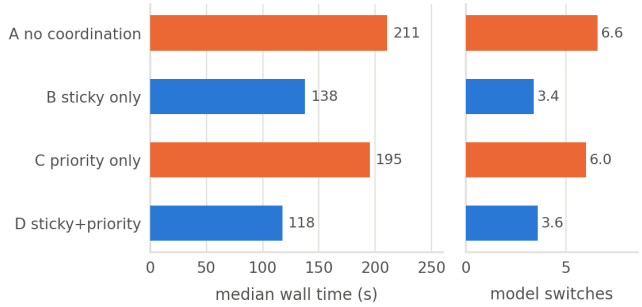


Fig. 4. Four-way ablation, five trials per condition. Blue marks the conditions that include sticky affinity. Left: median wall time across all agents. Right: mean model switches per trial.

Affinity is what stops the thrashing, and priority is not. Affinity on its own takes mean switches from 6.6 down to 3.4, effectively matching the full policy's 3.6, while priority on its own leaves them at 6.0. The efficiency comes from residency, in other words, and not from ordering. Priority does protect interactive agents, but only with affinity underneath it. Affinity alone takes the coding agent from a 201.6 s median to 113.4 s; adding priority takes it to 38.6 s. Priority without affinity is the bad case, improving total time by 7% while driving the lowest-priority agent from 76.3 s to 195.2 s, the worst figure any agent recorded in any condition. On confidence: the coordinated-against-uncoordinated comparison is clean, five of five trials non-overlapping. The four-way decomposition points the same way on every metric but its trial ranges overlap, so we would ask readers to take the pattern as supported and the effect sizes as approximate.

E. End-to-end, through unmodified third-party agents

The test we consider decisive is simple. Point a third-party agent CLI at the engine and run a task. Point the same CLI at Ramer and run the same task. Verify both outcomes from outside (Fig. 5).

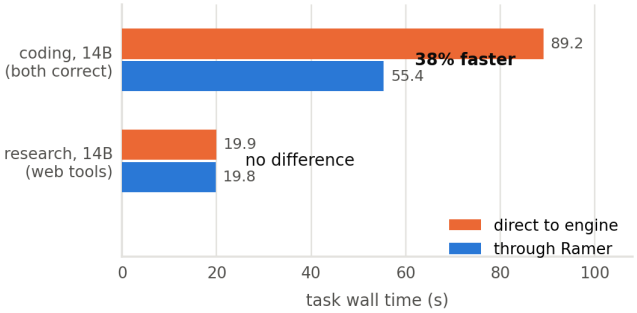


Fig. 5. Identical tasks through an unmodified third-party agent. Both coding runs produced correct, externally verified output. The research task is a deliberate contrast, one where tool and network latency dominate.

A coding task on the 14B model: 55.4 s against 89.2 s, both correct, a 38% improvement. A research task that had to search the web and read pages: 19.8 s against 19.9 s, which is no improvement at all, because the wall time there is network and tool latency and no inference-side change compresses that. A coding task on the 32B Q2_K model: neither condition succeeded, because placement cannot rescue a model that is not up to the task. Two of the three are negative, and they line up with independent characterizations reporting tool latency as a large share of agentic execution [38], [39].

F. Where our own metric misleads

The 27B IQ2_XXS model, at its measured placement, finished a real bug-fixing task correctly and verifiably, in 313.5 s. The 14B K-quant model finished the identical task in 157.8 s while decoding at a fifth of the speed. The transcript explains it. Several turns produced a single token in 126, 43 and 41 seconds, and the opening turn alone ate 126 of the 313.5. Time-to-first-token was the bottleneck, and Algorithm 1 times bounded decode, which never sees prefill. We cannot say from one run whether the cost is intrinsic to IQ-quant dequantization or particular to this quantization and context pairing, so we do not say it. What we can say is that this is a hole in our own method: a large decode gain need not become a proportionate end-to-end gain when an agent pays fresh prefill on every turn.

VII. DISCUSSION AND LIMITATIONS

A. Overlap with existing work

Choosing a layer count by measurement instead of estimate is not offered here as novel. Practitioners already sweep the parameter with the engine's own benchmarking tools [7], and the upstream fitting facility now fits the same parameters from inside the engine [6]. Proxy-based model swapping and single-machine residency management both exist already [8], [9]. On-device profiling at deployment time for memory-constrained inference is prior work [18]. What we do claim is the coupling of placement validity to context length, the mechanism that enforces it, the measured non-determinism that pushes the defence from design time to run time, the policy ablation in Section VI-D, and the combination of these in a layer that needs no change to the engine.

One lever stays out of reach. Keeping shared attention resident while routing individual experts to the CPU is a larger win than the dense-layer split, and the API we build on does not expose it. We confirmed that three ways, one of them a sweep on a real MoE model whose VRAM scaled in smooth per-block steps with no expert awareness anywhere in it. Getting at that lever means driving a different backend.

B. Threats to validity

One card, one driver, one operating system, and every number above comes from it. The oversubscription behaviour in particular is a property of the Windows display driver [40] and we have not tested its shape elsewhere. Context coupling rests on arithmetic that ought to generalize, but ought is not the same as shown. The failure in Section III-B is a single incident, offered with its mechanism measured and not as a controlled comparison. The ablation runs five trials per condition and the ranges overlap. The agentic comparisons are single runs over few tasks. Switch counts come from 0.5 Hz polling. The VRAM figures are the engine's own and, per Section III-C, can exceed physical capacity, so they describe memory requested and not memory resident. We did not instrument power or temperature, so we cannot say whether a denser placement changes the energy or thermal profile, which matters on the edge devices Section II-A surveys. Our comparison point is the engine's automatic placement, which is what users actually get, and not an expert hand-tuned configuration, which a few users do produce.

C. Future work

Algorithm 1 ought to time time-to-first-token alongside decode and weigh the two by the shape of the workload in front of it, which would have caught Section VI-F. Working out what separates smooth degradation from collapse above the ceiling would turn our runtime check into a design-time guarantee. And an importance signal of the sort MCAP produces [18] composes naturally with a measured budget, the two deciding which layers to demote and how many will fit; we think that combination is the most promising thing to try next.

VIII. CONCLUSION

Local inference engines treat weight placement as a property of a model and a device, and select it from a conservative static estimate. On the machine studied here that estimate cost between 6.7% and 402% of achievable decode throughput across three models, a spread no fixed margin covers. Placement is also a property of the context window, because cache and weights draw on one budget, which makes reuse across context lengths a correctness fault rather than a missed optimization; it produced a silent 5.9-fold collapse whose cause we confirmed by measuring memory directly. And exceeding physical memory turned out to be necessary but not sufficient for collapse, which removes the design-time defence and leaves a runtime one.

Ramer puts these together in a layer that needs no change to the inference engine: selection by measurement, keyed to

context, re-checked while serving, with a residency policy for concurrent agents whose two mechanisms we separated by ablation. About the benefit we are deliberately narrow. It is proportional to how much of a workload is genuinely local decoding: 38% on a verified coding task, nothing on a task ruled by tool latency, and nothing whatever for a model that was never going to finish the job. Reporting the second and third is what makes the first worth reading.

ACKNOWLEDGMENT

A generative AI system was used for literature search and for drafting and editing the manuscript text. The research question, experimental design, all measurements, and the analysis and conclusions drawn from them are the authors' own. The authors have verified every number, claim and citation in this paper and take full responsibility for its content.

REFERENCES

[1] K. Liu *et al.*, "Low-bit model quantization for deep neural networks: A survey," arXiv:2505.05530, 2025.

[2] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," arXiv:2210.17323, 2022.

[3] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for LLM compression and acceleration," arXiv:2306.00978, 2023.

[4] G. Xiao *et al.*, "SmoothQuant: Accurate and efficient post-training quantization for large language models," arXiv:2211.10438, 2022.

[5] Valve Corporation, "Steam hardware & software survey: August 2026," 2026. [Online]. Available: https://store.steampowered.com/hwsurvey/. Accessed: Sep. 30, 2026.

[6] J. Gaessler, "Automation for GPU layers, tensor split, tensor overrides, and context size (with MoE optimizations)," llama.cpp discussion #18049, Dec. 15, 2025. [Online]. Available: https://github.com/ggml-org/llama.cpp/discussions/18049. Accessed: Sep. 30, 2026.

[7] ggml-org, "llama.cpp: llama-bench and llama-sweep-bench." [Online]. Available: https://github.com/ggml-org/llama.cpp. Accessed: Sep. 30, 2026.

[8] llama-swap, "Reliable model swapping for any local OpenAI-compatible server." [Online]. Available: https://github.com/mostlygeek/llama-swap. Accessed: Sep. 30, 2026.

[9] Ollama, "Request scheduling and runner management." [Online]. Available: https://github.com/ollama/ollama/blob/main/docs/faq.md. Accessed: Sep. 30, 2026.

[10] NVIDIA, "GeForce RTX 5060 specifications." [Online]. Available: https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5060/. Accessed: Sep. 30, 2026.

[11] S. Laskaridis *et al.*, "MELT: Mobile evaluation of language transformers," in *Proc. MobiCom*, 2024.

[12] MLC team, "MLC-LLM: Universal LLM deployment engine with machine learning compilation." [Online]. Available: https://github.com/mlc-ai/mlc-llm. Accessed: Sep. 30, 2026.

[13] D. Xu *et al.*, "Fast on-device LLM inference with NPUs," in *Proc. ASPLOS*, 2025, doi: 10.1145/3669940.3707239.

[14] Z. Xue, Y. Song, Z. Mi, L. Chen, Y. Xia, and H. Chen, "PowerInfer-2: Fast large language model inference on a smartphone," arXiv:2406.06282, 2024.

[15] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, "EdgeMoE: Fast on-device inference of MoE-based large language models," *IEEE Trans. Mobile Comput.*, 2025, pp. 7059–7073.

[16] K. Alizadeh *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," arXiv:2312.11514, 2023.

[17] Y. Song *et al.*, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[18] A. Das, "MCAP: Deployment-time layer profiling for memory-constrained LLM inference," arXiv:2604.21026, 2026.

[19] Z. Wang *et al.*, "Speculative decoding with CTC-based draft model for LLM inference acceleration," in *Proc. NeurIPS*, 2024.

[20] "Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference," arXiv:2412.18934, 2024.

[21] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[22] "Rethinking key-value cache compression techniques for large language model serving," arXiv:2503.24000, 2025.

[23] "KV cache optimization strategies for scalable and efficient LLM inference," arXiv:2603.20397, 2026.

[24] LMCache Team, "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," Tech. Rep., 2025. [Online]. Available: https://lmcache.ai/tech_report.pdf. Accessed: Sep. 30, 2026.

[25] DeepSpeed Team, "ZeRO-Inference: Democratizing massive model inference," 2022. [Online]. Available: https://www.deepspeed.ai/2022/09/09/zero-inference.html. Accessed: Sep. 30, 2026.

[26] Y. Sheng *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, 2023.

[27] K. Kamahori *et al.*, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *Proc. ICLR*, 2025.

[28] T. Xu, L. Xue, Z. Lu, A. Jackson, and L. Mai, "MoE-Gen: High-throughput MoE inference on a single GPU with module-based batching," arXiv:2503.09716, 2025.

[29] "MoBiLE: Efficient mixture-of-experts inference on consumer GPU with mixture of big little experts," arXiv:2510.12357, 2025.

[30] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *Proc. OSDI*, 2022.

[31] A. Agrawal *et al.*, "SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills," arXiv:2308.16369, 2023.

[32] A. Agrawal *et al.*, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. OSDI*, 2024.

[33] S. Yu *et al.*, "Prism: Cost-efficient multi-LLM serving via GPU memory ballooning," in *Proc. OSDI*, 2026.

[34] kvcached, "Virtual-memory ballooning for LLM serving." [Online]. Available: https://github.com/ovg-project/kvcached. Accessed: Sep. 30, 2026.

[35] Y. Fu *et al.*, "ServerlessLLM: Low-latency serverless inference for large language models," in *Proc. OSDI*, 2024.

[36] "Bench360: Benchmarking local LLM inference from 360 degrees," arXiv:2511.16682, 2025.

[37] X. Qiu, C. Xu, B. Su, Z. Chen, R. Xu, and S. Dai, "GGUF-metadata prediction of single-sequence llama.cpp throughput across three systems," arXiv:2609.14864, 2026.

[38] C. Chang *et al.*, "From LLM inference to agentic workloads: Characterization and implications for serving systems," arXiv:2608.15127, 2026.

[39] Y. Yuan, A. Nayak, S. Kundu, and N. Talati, "Agentic AI workload characterization," arXiv:2605.26297, 2026.

[40] NVIDIA, "System memory fallback for Stable Diffusion," NVIDIA support answer 5490. [Online]. Available: https://nvidia.custhelp.com/app/answers/detail/a_id/5490. Accessed: Sep. 30, 2026.

[41] NVIDIA, "Improving GPU memory oversubscription performance," NVIDIA Technical Blog. [Online]. Available: https://developer.nvidia.com/blog/improving-gpu-memory-oversubscription-performance/. Accessed: Sep. 30, 2026.